
TECHNICAL REPORT

Forecasting Hourly Household Electricity Consumption

*A Leakage-Safe Comparison of Classical,
Tree-Based, and Sequence Models*

Rolling-origin cross-validation • Diebold–Mariano significance testing
Free-weather augmentation • Business-value analysis for Enel Global ICT

TEAM

Thierry Ishimwe (*group delegate*)

`ishimwe.thierry@studenti.luiss.it`

Linda Carla Zorzoli

`zorzoli.lindacarla@studenti.luiss.it`

Cesar Dushimimana

`cesar.dushimimana@studenti.luiss.it`

Mariavittoria Giurato

`m.giurato@studenti.luiss.it`

COURSE & PARTNER

Course

Artificial Intelligence Techniques — LUISS Guido Carli

Industry partner

Enel Global ICT

Date

May 2026

1 Introduction

This project develops an hourly electricity-consumption forecasting pipeline for a single household and uses it to compare four model families on a uniform protocol. The case study is the UCI *Individual Household Electric Power Consumption* dataset [1]: 2,075,259 minute-level active-power measurements from a household in Sceaux (Paris area), France, recorded between December 2006 and November 2010 and resampled to hourly mean for modelling (34,000 hourly observations). The goal is to forecast next-hour `Global_active_power` in a leakage-safe way, and to evaluate four model families — SARIMA, XGBoost, LSTM, and GRU — against three naive persistence baselines on six rolling-origin folds.

The headline finding is that all four real model families cluster within a 3% MAE band (0.337–0.345 kW); the top three (XGBoost, GRU, LSTM) are statistically indistinguishable under the Diebold–Mariano test; and augmenting the leaderboard winner with four free Paris–Montsouris weather columns reduces cross-fold MAE by 0.77% — a gain concentrated on the heating-season folds, where forecasting hurts most. For Enel, the implication is that on residential consumption at this resolution, the next gain comes from *better input information*, not from a different architecture.

2 Methods

Figure 1 summarises the end-to-end pipeline. Each block below traces back to a design decision motivated by an EDA finding or a leakage-safety constraint — the rest of this section explains the *why* behind each.

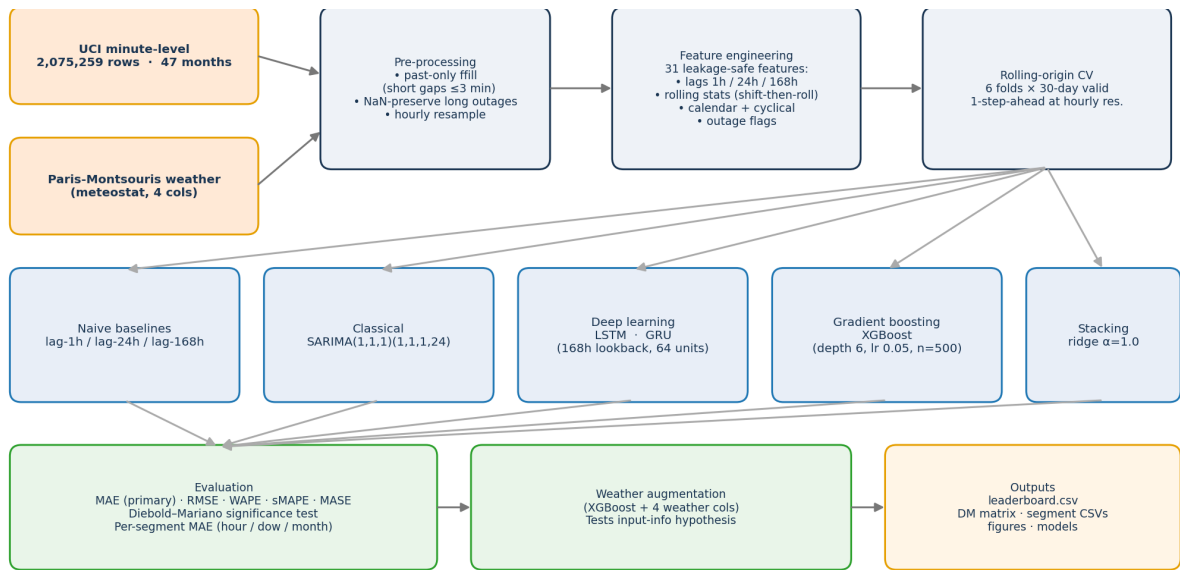


Figure 1. End-to-end leakage-safe forecasting pipeline. Data collection feeds a two-stage cleaning policy, a leakage-safe feature registry, and a rolling-origin cross-validation harness shared by every model family.

2.1 Data collection

The primary source is the UCI dataset [1], SHA-256-pinned in the project configuration and integrity-verified on download. Weather data was collected separately from station Paris–Montsouris (NOAA / Météo-France, via the *meteostat* client [2]). **Why this station:** it lies 5.2 km from Sceaux — the closest publicly available source with continuous hourly records covering the full study period — close enough that its temperature trace is operationally representative of the household. **Why hourly resampling:** European electricity markets bid in one-hour blocks, and the EDA showed that minute-level variability above the hourly aggregate is dominated by appliance-switching noise rather than predictable signal — modelling at the minute frequency would inflate evaluation error without improving operational utility.

2.2 Data pre-processing: how we cleaned the dataset

The EDA found 25,979 missing minute rows (1.25%) across 71 distinct gap segments, bimodally distributed: 54 short gaps (≤ 3 min, 72 minutes total) and 17 long outages (> 3 min, 25,907 minutes total). The longest outage spans five days in August 2010, coinciding with the French summer-vacation period. This bimodality drives a two-stage policy rather than a single global rule.

- **Short gaps** → **past-only forward-fill**. These look like sensor-blip noise. Each missing value is filled with the last observed value, with no interpolation. *Why not bilateral interpolation?* A bilateral fill would use a future observation $y[t+k]$ to impute $y[t]$, which then flows into lag and rolling features at later timestamps — an indirect form of target leakage. Forward-only fill is the only choice that survives the “information available at forecast time” test.
- **Long outages** → **preserved as NaN plus an explicit indicator**. Multi-day outages encode a structural reality: the household is offline. Imputing them with synthetic values would feed fictional consumption into training. Instead they are kept as NaN, dropped per fold, and flagged with a boolean `is_outage_gap` feature, so models can see the outage rather than be silently fed an imputed value.
- **Hourly resampling preserves the indicator**. Numeric columns are aggregated by mean; the outage flag is aggregated by max — an hour is flagged if any of its 60 source minutes fell inside a long gap.

These three policies are guarded by unit tests (`tests/unit/test_missing_value_policy.py`), so they cannot regress silently.

2.3 Feature engineering

The feature matrix is **leakage-safe by construction**. Every feature is registered with a *leakage_safe* flag and a written rationale; the pipeline refuses to emit any column flagged unsafe. **Why this matters:** six UCI columns (`Voltage`, `Global_intensity`, `Global_reactive_power`, `Sub_metering_1/2/3`) are same-timestamp physical components of the target ($P = V \cdot I$; the sub-meters sum into the total). If they entered the matrix, a model would *reconstruct* the target instead of forecasting it. The registry blocks them mechanically.

The remaining 31 features fall into five families, each motivated by a specific EDA finding: **target lags** at 1 h, 24 h, and 168 h — chosen because the differenced-series ACF showed peaks at lag 168 (+0.21, weekly) and lag 24 (+0.20, daily); **rolling statistics** (mean, std, min, max) over 3-, 24-, and 168-hour windows, computed as `shift(1).rolling(W)` so the window at time t covers strictly past values; **calendar features** (hour, day-of-week, month, `is_weekend`, `is_french_holiday`), because the EDA surfaced strong daily and weekly cycles; **cyclical sine/cosine encodings** of the calendar dimensions, so tree-based models handle the midnight and end-of-week wrap-around without explicit interaction terms; and **outage indicators** carried over from the pre-processing stage.

2.4 Models considered and why

Eight models span four families. The selection is not exhaustive but hypothesis-driven: each family tests a different claim about which class of structure dominates the signal. SARIMA is the classical baseline and tests whether linear AR/MA/seasonal structure alone is sufficient. LSTM [4] and GRU [5] test whether end-to-end sequence learning extracts more from the raw 168-hour history than the engineered features do; running both cells lets us compare LSTM’s extra output gate against GRU’s simpler cell on an otherwise identical architecture. XGBoost [3] tests whether non-linear interactions over the 31 engineered features (e.g. `hour × is_weekend`) beat both the linear and the sequence approaches; it also provides transparent gain-based feature importance — an interpretability bonus the deep-learning models cannot offer. Stacking tests whether the four base models make decorrelated errors. XGBoost + weather tests the “input information vs. architecture” hypothesis directly. The three persistence baselines anchor the leaderboard.

Table 1. Model line-up with role tags. *Brief* items satisfy the project requirements directly; *Additional* items go beyond the minimum, with the value they add stated inline.

Model	Family	Role
3 naive baselines (lag-1h, -24h, -168h)	Persistence	<i>Brief:</i> baselines for comparison
SARIMA(1,1,1)(1,1,1) ₂₄	Classical statistical	<i>Brief:</i> ≥ 1 classical model
LSTM (168 h lookback, 64 units)	Deep-learning sequence	<i>Brief:</i> ≥ 1 deep-learning model
GRU (168 h lookback, 64 units)	Deep-learning sequence	<i>Additional:</i> within-DL comparison
XGBoost (depth 6, lr 0.05, $n=500$)	Gradient-boosted trees	<i>Additional:</i> 4th family, transparent feature importance
Stacking (ridge $\alpha=1.0$)	Ensemble	<i>Additional:</i> tests base-model error decorrelation
XGBoost + 4 weather features	Tree + exogenous	<i>Additional:</i> tests the input-information bottleneck

2.5 Evaluation protocol and metrics

We use rolling-origin cross-validation: 6 folds, each training on 365 days and validating on the following 30 days, with folds sliding forward in 30-day increments. **Why rolling-origin and not random k -fold?** Random splits would let a fold’s training set include observations chronologically after its validation set — impossible at the moment a real forecast is made, and a source of inflated accuracy. **Why 6 folds $\times$ 30 days?** Each validation window of 720 hours is large enough for the Diebold–Mariano test to have power ($n \approx 700$ paired predictions per fold), and six distinct slides cover the full annual cycle without overlap. **Why 1-step-ahead?** It establishes the leakage boundary cleanly: each prediction uses information available through hour $t-1$ only. Multi-step horizons (e.g. 24-step day-ahead) are operationally relevant but complicate the leakage discussion and are recorded as future work.

Metrics. MAE is the primary metric — robust to the right-skewed target (skewness ≈ 1.5); RMSE serves as a peak-error guardrail; WAPE and sMAPE give scale-free views; and MASE (seasonal period 24) anchors against persistence — a value below 1 means the model beats the in-sample daily-naive baseline.

Baselines. Three persistence models — `naive_lag_1h`, `naive_lag_24h`, `naive_lag_168h` — establish the floor any real model must beat to deserve a place on the leaderboard. Their three lag horizons probe different periodicities (hour-to-hour, daily, weekly).

Hyperparameter tuning was not performed in this iteration: each model uses documented defaults from its published literature (XGBoost depth 6 / lr 0.05 / 500 trees; LSTM and GRU with 64 hidden units, 168-hour lookback, Adam, 25 epochs; SARIMA order chosen from ADF/KPSS tests plus STL decomposition). An Optuna-based search is a planned follow-up.

3 Results and Discussion

3.1 Technical results

The leaderboard (Table 2) confirms that the four real model families cluster within a narrow band. XGBoost wins at cross-fold mean MAE 0.337 kW, followed by GRU (0.339), LSTM (0.341), and SARIMA (0.345) — a maximum spread of 3% across the cluster. All four beat the strongest naive baseline (`naive_lag_1h` at 0.380) by 9–12%, and every real model returns $MASE < 1$, confirming it is genuinely forecasting rather than coincidentally tracking persistence.

Diebold–Mariano significance test. Aggregate MAE answers “which model has the lowest average error?” but not “is that difference statistically real?”. We ran pairwise Diebold–Mariano

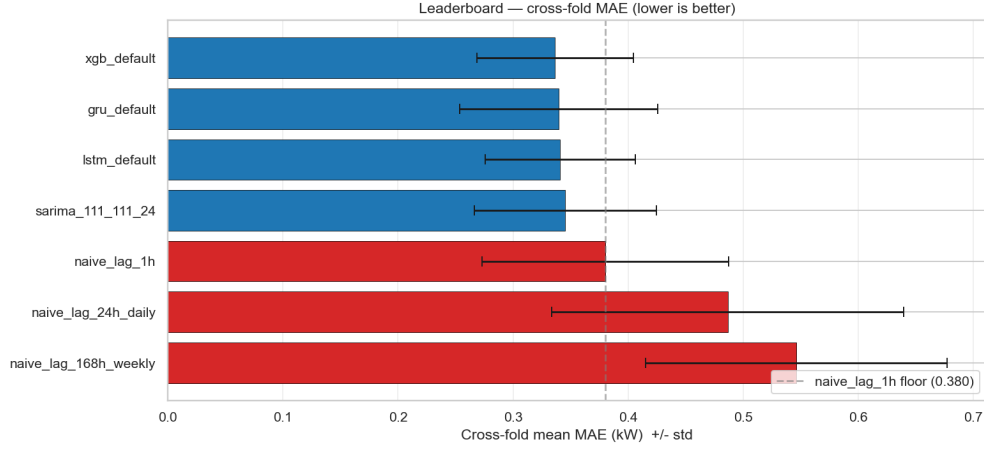


Figure 2. Cross-fold mean MAE per model with $\pm 1\sigma$ error bars across the six rolling-origin folds. Red bars are persistence baselines; blue bars are the real model families. The dashed line marks the strongest naive floor (0.380 kW).

Table 2. Leaderboard — cross-fold mean $\pm$ standard deviation across the six folds. The four real model families cluster within 3% MAE.

Model	MAE	RMSE	MASE
XGBoost	0.337 $\pm$ 0.068	0.468 $\pm$ 0.093	0.573 $\pm$ 0.119
GRU	0.339 $\pm$ 0.086	0.475 $\pm$ 0.103	0.578 $\pm$ 0.149
LSTM	0.341 $\pm$ 0.065	0.470 $\pm$ 0.091	0.580 $\pm$ 0.114
SARIMA	0.345 $\pm$ 0.079	0.490 $\pm$ 0.097	0.588 $\pm$ 0.137
Naive lag-1h	0.380 $\pm$ 0.107	0.563 $\pm$ 0.135	0.647 $\pm$ 0.186
Naive lag-24h	0.487 $\pm$ 0.153	0.715 $\pm$ 0.180	0.828 $\pm$ 0.264
Naive lag-168h	0.546 $\pm$ 0.131	0.777 $\pm$ 0.170	0.929 $\pm$ 0.230

tests [6] with the Harvey–Leybourne–Newbold small-sample correction [7] on $n \approx 4,100$ paired absolute-error predictions (Table 3). Only XGBoost vs. SARIMA ($p = 0.022$) clears the $\alpha = 0.05$ bar. XGBoost vs. LSTM ($p = 0.33$) and XGBoost vs. GRU ($p = 0.42$) are not statistically distinguishable — the top three are a statistical tie. The added value of the DM test is that it answers a question MAE alone cannot: whether the leaderboard order is real or within noise.

Table 3. Pairwise Diebold–Mariano p -values between the four real model families. Values below the $\alpha = 0.05$ threshold (shaded) indicate a statistically significant difference in predictive accuracy.

	XGBoost	SARIMA	LSTM	GRU
XGBoost	—	0.022	0.330	0.415
SARIMA	0.022	—	0.108	0.073
LSTM	0.330	0.108	—	0.747
GRU	0.415	0.073	0.747	—

Per-segment analysis. MAE decomposed by hour-of-day, day-of-week, and validation month (Figure 3) surfaces operationally important structure that aggregate MAE hides. Every model shares the same hour-of-day shape: easy at 03–06 h (MAE ≈ 0.17), hard at 18–21 h (MAE ≈ 0.45); XGBoost wins the quiet hours, while GRU wins the peak window. The day-of-week effect is modest, with weekends only slightly harder than weekdays. The monthly pattern is dominated by two regimes: the *grandes vacances* (August, MAE 0.21–0.25, an empty household) and the regulated

période de chauffe (October, MAE 0.42–0.44, the heating ramp).

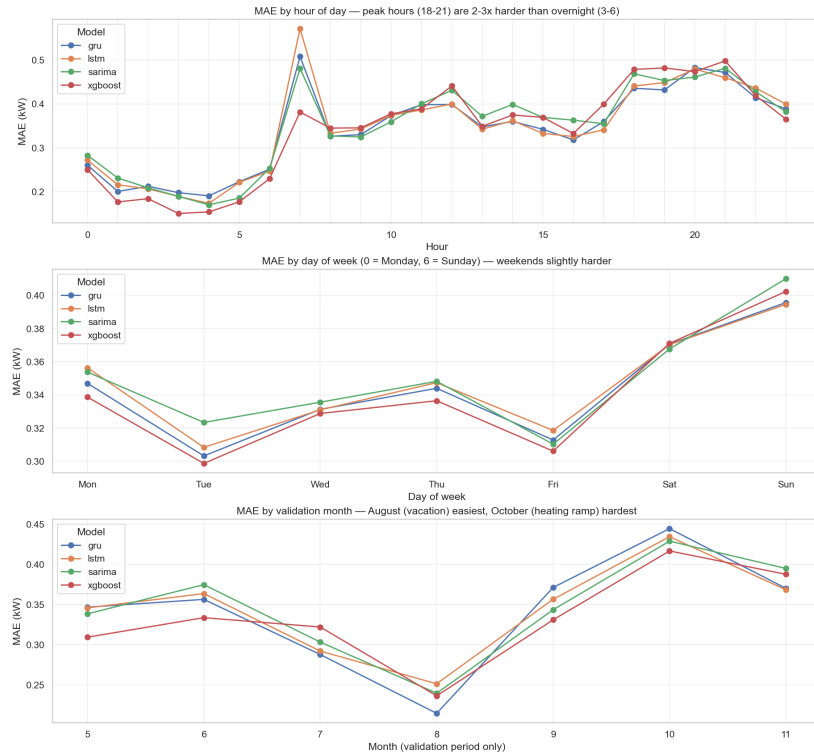


Figure 3. Per-segment MAE by model: by hour-of-day (top), day-of-week (middle), and validation month (bottom). Peak evening hours and the October heating ramp drive the bulk of every model’s error budget.

Weather augmentation. Residential electricity consumption in France is dominated by electric heating (roughly 30% of the national housing stock, per ADEME). Stacking confirmed that more model variety would not break the 3% MAE cluster, so we tested the complementary hypothesis — that the binding constraint is input information. XGBoost was retrained on the feature matrix augmented with four hourly Paris–Montsouris weather columns (temperature, relative humidity, wind speed, and a strictly past 24-hour-lagged temperature). The experiment adds value in three ways: (i) it puts a measured number against the “input information vs. architecture” hypothesis; (ii) the four columns are free, leakage-safe, and cheap to integrate; (iii) the per-fold pattern tells us *when* weather helps, not merely *whether* it helps.

Table 4. Weather-augmented XGBoost vs. baseline XGBoost. The -0.77% mean is small but signed in the predicted direction; the improvement concentrates on the heating-season folds (2–6), while the easiest summer fold gains nothing.

Fold	Baseline MAE	+ Weather MAE	Δ MAE	Δ %
1	0.3351	0.3359	+0.0008	+0.24%
2	0.3324	0.3269	−0.0055	−1.65%
3	0.2147	0.2122	−0.0025	−1.16%
4	0.3352	0.3320	−0.0032	−0.95%
5	0.4024	0.3999	−0.0025	−0.62%
6	0.3994	0.3967	−0.0027	−0.68%
Mean	0.3365	0.3339	−0.0026	−0.77%

The cross-fold mean MAE improves from 0.3365 to 0.3339, a -0.77% gain (Table 4). The per-fold pattern is more informative than the headline: improvements concentrate on the harder folds (2–6:

late winter, the spring shoulder, the October heating ramp), while the easiest summer fold gains nothing. This is exactly the predicted pattern — weather information matters when consumption is temperature-driven, not when the household sits at a flat low level.

3.2 Project-oriented (business-value) implications

For Enel Global ICT, three implications follow from the technical results.

- **Architecture choice is a near-decision on this dataset.** XGBoost, GRU, and LSTM are statistically tied. The choice among them should be driven by non-MAE factors — inference latency, retraining cost, interpretability, and peak-hour accuracy, a real driver of imbalance-market penalties for grid operators.
- **Peak-hour accuracy and overall MAE rank models differently.** XGBoost minimises overall MAE, but GRU minimises 18–21 h MAE. A deployment weighted toward peak-hour cost should prefer GRU; one weighted toward overall accuracy can keep XGBoost. The aggregate leaderboard hides this trade-off — the per-segment analysis surfaces it.
- **Free weather data yields a real but modest improvement.** Four Paris–Montsouris columns reduce XGBoost MAE by 0.77%, with the gain concentrated on heating-season folds. For a fleet-level residential forecast this is a non-trivial reduction in imbalance-market exposure during winter. The deeper implication is that better input information — richer weather features, occupancy proxies, calendar-aware regressors for SARIMA — is where additional gains live, not in trying new model architectures.

4 Conclusions

- **The pipeline is leakage-safe by construction.** A feature registry mechanically excludes same-timestamp components of the target, short-gap imputation is past-only, and two regression tests guard both policies on every commit.
- **XGBoost wins the leaderboard** at cross-fold MAE 0.337 kW, beating SARIMA significantly (Diebold–Mariano $p = 0.022$) but not beating LSTM ($p = 0.33$) or GRU ($p = 0.42$). The top three are statistically tied.
- **GRU is competitive with LSTM at lower complexity.** Across all six folds, GRU’s MAE (0.339) is essentially tied with LSTM’s (0.341); the simpler cell captures the relevant sequential structure just as well.
- **Stacking confirms the bottleneck is input information, not model variety.** A ridge ensemble over the four base models does not break the 3% cross-fold cluster — the base-model errors are too correlated.
- **Weather augmentation validates the input-information hypothesis empirically.** Adding four weather columns to XGBoost reduces cross-fold MAE by 0.77%, concentrated on the heating-season folds.
- **Operationally, the best model depends on the metric.** XGBoost wins overall MAE; GRU wins peak-hour MAE. A grid operator weighting peak-hour accuracy heavily should prefer GRU.

Future work

Three extensions follow naturally, each scoped by what it would add rather than what is missing.

- **Extend weather augmentation to SARIMA, LSTM, and GRU.** The “input information vs. architecture” hypothesis was tested on the leaderboard winner only. Applying the same four weather columns to the other three families would show whether the -0.77% gain is a single-model effect or generalises — and would settle whether a non-XGBoost deployment is also worth supplementing with weather data.
- **Switch from 1-step-ahead to 24-step day-ahead forecasting.** The current protocol predicts hour t from information through hour $t-1$. Enel’s market-bid horizon is a full day, made from a single daily information cutoff. The protocol switch would quantify the gap between

methodology MAE and the operationally realistic MAE that drives imbalance-penalty exposure.

- **Hyperparameter optimisation via Optuna.** All models ran on documented defaults. A Bayesian-TPE search across the model-specific hyperparameter spaces (XGBoost depth / learning rate / tree count; LSTM and GRU hidden units / dropout; SARIMA AR and MA orders) would tell us whether the 3% MAE cluster is a true ceiling or an artefact of un-tuned defaults.

References

- [1] G. Hébrail and A. Bérard. *Individual Household Electric Power Consumption* [Data set]. UCI Machine Learning Repository, 2012.
- [2] Meteostat. *Meteostat Python Library*. dev.meteostat.net (accessed May 2026).
- [3] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [4] S. Hochreiter and J. Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [5] K. Cho et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. In *Proc. EMNLP*, pp. 1724–1734, 2014.
- [6] F. X. Diebold and R. S. Mariano. Comparing Predictive Accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263, 1995.
- [7] D. I. Harvey, S. J. Leybourne, and P. Newbold. Testing the Equality of Prediction Mean Squared Errors. *International Journal of Forecasting*, 13(2):281–291, 1997.

Appendix A – Code Description

The source code in `src.zip` is organised as numbered scripts run in order, supported by a small Python package. The pseudocode below traces the end-to-end pipeline.

01_build_features.py

```
raw = load_uci('data/raw/data.txt')
short_gaps, long_gaps = classify_missing(raw)
cleaned = forward_fill(raw, max_gap = 3min) # past-only
hourly = resample(cleaned, '1h', mean)
features = build(target = hourly.Global_active_power,
                 lags = [1, 24, 168],
                 rolling = [3, 24, 168],
                 calendar = ['hour', 'dow', 'month', 'is_weekend', 'is_french_holiday'],
                 cyclical = ['hour', 'dow', 'month']) # sin/cos
save(features, 'data/features.parquet')
```

02_fetch_weather.py

```
weather = meteostat.Hourly('07156', start, end).fetch() # Paris-Montsouris
save(weather[['temp', 'rhum', 'wspd']] + lag24(temp),
      'data/paris_montsouris_weather.parquet')
```

04-08_run_*.py

```
for fold in rolling_origin_splits(features.index, n = 6, valid_days = 30):
    Xt, yt = features.loc[fold.train], target.loc[fold.train]
    Xv, yv = features.loc[fold.valid], target.loc[fold.valid]
    model = Forecaster(**hyperparams)
    model.fit(Xt, yt)
    preds = model.predict_one_step_ahead(Xv)
    save(preds, f'results/_predictions_fold{fold.id}.csv')
    save(metrics(yv, preds), f'results/_fold_results.csv')
```

09_run_stacking.py

```
base = load_predictions(['xgboost', 'sarima', 'lstm', 'gru'], folds = 1.5)
meta = Ridge(alpha = 1.0).fit(base.X, base.y)
eval(meta, predictions(folds = 6))
```

10_run_dm_test_and_segments.py

```
p_values = pairwise_dm_test(predictions_folds_1_6)
segment_mae = group_by(['hour', 'dow', 'month'], MAE)
save(p_values, 'results/dm_test_pvalues.csv')
```

11_aggregate_leaderboard.py

```
combine fold_results CSVs → mean ± std per model → results/leaderboard.csv
```

12_build_technical_report.py / 13_build_presentation.py

```
load tracked artifacts, render PDF / PPTX
```

Appendix B – Author Contribution and Generative AI Statement

B.1 Author Contribution (CRediT)

Using the CRediT taxonomy, contributions across the four authors are as follows.

- **Linda Carla Zorzoli & Mariavittoria Giurato** — *Investigation* (exploratory data analysis); *Resources* (research on the documented French and European events explaining the dataset outages); *Validation* (review of the submission materials against the project rules).
- **Thierry Ishimwe & Cesar Dushimimana** — *Methodology* and *Software* (feature engineering, modelling, evaluation pipeline, reporting tooling); *Writing — original draft* and *Writing — review & editing*.
- *Conceptualization* and *Project administration* were shared across all four authors.

B.2 Generative AI Statement

We used Anthropic’s Claude during development for: (a) extensive leakage checking — auditing the feature pipeline for same-timestamp target components, verifying that lag and rolling features are strictly past-only, and surfacing the bilateral-interpolation leak in the missing-value policy, which we then fixed; (b) brainstorming the registry-based leakage-prevention design and the past-only-fill regression test; (c) reviewing wording across the notebook, this technical report, and the presentation. All AI-generated code was reviewed, executed, and validated against tests. All design decisions and final wording are owned and understood by the authors.